TP VitaSport : mes réponses

Dorian Pachot · BTS SIO SLAM, 2e année · Bases de données avancées · parties 1 à 5 (octobre 2026)

PARTIE 1 — Rappels SQL

1.1 Créer la base

```
CREATE DATABASE vitasport_decouverte
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
USE vitasport_decouverte;
```

1.2 Table formule

```
CREATE TABLE formule (
  id_formule     INT           AUTO_INCREMENT PRIMARY KEY,
  libelle        VARCHAR(30)   NOT NULL UNIQUE,
  prix_mensuel   DECIMAL(6,2)  NOT NULL,
  nb_resa_hebdo  TINYINT       NULL
) ENGINE = InnoDB;
```

1.3 Table membre

```
CREATE TABLE membre (
  id_membre      INT AUTO_INCREMENT PRIMARY KEY,
  nom            VARCHAR(50)  NOT NULL,
  prenom         VARCHAR(50)  NOT NULL,
  email          VARCHAR(100) NOT NULL UNIQUE,
  date_naissance DATE         NOT NULL,
  id_formule     INT          NOT NULL,
  CONSTRAINT fk_membre_formule
    FOREIGN KEY (id_formule) REFERENCES formule(id_formule)
) ENGINE = InnoDB;

Question 1.3
-- Réponse : membre contient une clé étrangère qui pointe vers formule. On ne peut pas
-- référencer une table qui n'existe pas encore, donc formule (la table "parent") d'abord.
-- Pour supprimer, c'est l'ordre inverse : d'abord membre (l'enfant), puis formule.
```

1.4 Insérer les données

```
INSERT INTO formule (libelle, prix_mensuel, nb_resa_hebdo) VALUES
  ('Essentiel', 24.90, 2),
  ('Confort',   34.90, 4),
  ('Premium',   49.90, NULL),
  ('Étudiant',  19.90, 2);

INSERT INTO membre (nom, prenom, email, date_naissance, id_formule) VALUES
  ('Durand', 'Emma',   'emma.durand@mail.fr',   '2001-04-12', 4),
  ('Dubois', 'Lucas',  'lucas.dubois@mail.fr',  '1988-11-03', 2),
  ('Martin', 'Jade',   'jade.martin@mail.fr',   '1995-07-21', 1),
  ('Petit',  'Hugo',   'hugo.petit@mail.fr',    '2004-01-30', 4),
  ('Leroy',  'Louise', 'louise.leroy@mail.fr',  '1979-09-15', 3),
  ('Duval',  'Nathan', 'nathan.duval@mail.fr',  '1999-02-08', 1),
```

```
  ('Garcia', 'Alice',  'alice.garcia@mail.fr',  '1992-06-17', 2),
  ('Roux',   'Léo',    'leo.roux@mail.fr',      '2002-12-01', 1);
```

1.5 Provoquer des erreurs

```
INSERT INTO membre (nom, prenom, email, date_naissance, id_formule)
VALUES ('Test', 'Fk', 'test.fk@mail.fr', '2000-01-01', 99);
-- Erreur 1452 : Cannot add or update a child row: a foreign key constraint fails
(fk_membre_formule)

INSERT INTO membre (nom, prenom, email, date_naissance, id_formule)
VALUES ('Doublon', 'Emma', 'emma.durand@mail.fr', '2000-01-01', 1);
-- Erreur 1062 : Duplicate entry 'emma.durand@mail.fr' for key 'email'
-- Explication : email est UNIQUE et cette adresse existe déjà (Emma Durand).
```

1.6 Interroger

```
a. SELECT * FROM membre;                                          -- 8 lignes
b. SELECT nom, prenom FROM membre ORDER BY nom, prenom;           -- 8
c. SELECT * FROM membre WHERE date_naissance >= '2000-01-01';     -- 3
d. SELECT * FROM formule WHERE prix_mensuel < 30 ORDER BY prix_mensuel ASC;   -- 2
e. SELECT * FROM membre WHERE nom LIKE 'D%';                      -- 3
f. SELECT COUNT(*) AS nb_membres FROM membre;                            -- 8
g. SELECT * FROM formule WHERE nb_resa_hebdo IS NULL;             -- 1 (Premium)
h. SELECT * FROM membre ORDER BY date_naissance DESC LIMIT 3;     -- Hugo, Léo, Emma
i.
   SELECT m.prenom, m.nom, f.libelle, f.prix_mensuel
   FROM membre m
   INNER JOIN formule f ON m.id_formule = f.id_formule
   ORDER BY f.prix_mensuel DESC;                                        -- 8
j.
   SELECT DISTINCT f.libelle
   FROM formule f
   INNER JOIN membre m ON m.id_formule = f.id_formule;                  -- 4
k.
   SELECT * FROM membre
   WHERE id_formule IN (1, 4)
     AND YEAR(date_naissance) BETWEEN 1995 AND 2002;                  -- 4
```

1.7 Faire évoluer la structure

```
a. ALTER TABLE membre ADD COLUMN telephone VARCHAR(15) NULL AFTER email;
b. ALTER TABLE formule ADD CONSTRAINT ck_formule_prix CHECK (prix_mensuel >= 0);
   -- vérification :
   UPDATE formule SET prix_mensuel = -5 WHERE id_formule = 1;
   -- Erreur 3819 : Check constraint 'ck_formule_prix' is violated.
c. ALTER TABLE membre MODIFY prenom VARCHAR(60) NOT NULL;
   -- (MODIFY réécrit toute la colonne : il faut remettre NOT NULL sinon on le perd)
```

1.8 Modifier des données

```
a.
   SELECT libelle, prix_mensuel AS ancien_prix, ROUND(prix_mensuel * 1.05, 2) AS nouveau_prix
   FROM formule
   WHERE libelle <> 'Étudiant';

   UPDATE formule
   SET prix_mensuel = ROUND(prix_mensuel * 1.05, 2)
```

```
   WHERE libelle <> 'Étudiant';
b.
   UPDATE membre SET telephone = '06 12 34 56 78'
   WHERE nom = 'Martin' AND prenom = 'Jade';
c.
   UPDATE membre
   SET id_formule = (SELECT id_formule FROM formule WHERE libelle = 'Confort')
   WHERE nom = 'Duval' AND prenom = 'Nathan';
```

1.9 Supprimer des données

```
a.
   DELETE FROM formule WHERE id_formule = 3;
   -- Erreur 1451 : Cannot delete or update a parent row: a foreign key constraint fails
(fk_membre_formule)
   -- Réponse : Louise Leroy a la formule Premium. Supprimer la formule laisserait
   -- un membre pointant vers une formule inexistante. Par défaut la FK est en RESTRICT ->
refus.
b.
   DELETE FROM membre WHERE nom = 'Roux' AND prenom = 'Léo';
c.
   START TRANSACTION;
   DELETE FROM membre;
   SELECT COUNT(*) FROM membre;   -- 0
   ROLLBACK;
   SELECT COUNT(*) FROM membre;   -- 7 (Léo Roux a été supprimé au b)

Question 1.9
-- Réponse : ROLLBACK annule tout ce qui a été fait depuis START TRANSACTION, la base revient
-- dans l'état d'avant. Habitude : avant un UPDATE/DELETE délicat, (1) faire d'abord un SELECT
-- avec le même WHERE pour voir les lignes touchées, (2) travailler dans une transaction et ne
-- faire COMMIT qu'après avoir vérifié le résultat.
```

1.10 Supprimer la structure

```
DROP TABLE formule;
-- Erreur 3730 : Cannot drop table 'formule' referenced by a foreign key constraint
-- 'fk_membre_formule' on table 'membre'.
-- Explication : membre dépend de formule. Il faut supprimer l'enfant d'abord :
DROP TABLE membre;
DROP TABLE formule;
```

PARTIE 2 — Modélisation Merise : VitaSport+

2.1 Lire le MCD existant

```
ACCUEILLIR (SALLE 0,n — SÉANCE 1,1)
- Une salle accueille 0 à n séances (une salle qui vient d'ouvrir n'a encore aucune séance).
- Une séance se déroule dans une et une seule salle.

HABILITER (COACH 0,n — ACTIVITÉ 1,n)
- Un coach est habilité à animer 0 à n activités (un coach qui vient d'être embauché peut ne
pas encore l'être).
- Une activité doit pouvoir être animée par au moins un coach habilité, éventuellement
plusieurs.

PARRAINER (réflexive sur ADHÉRENT : parrain 0,n — filleul 0,1)
- Un adhérent (parrain) peut parrainer 0 à n adhérents.
```

```
- Un adhérent (filleul) est parrainé par au plus un adhérent (0 s'il est venu seul).

RÉGLER (ABONNEMENT 1,n — PAIEMENT 1,1)
- Un abonnement est réglé par au moins un paiement (le 1er mois est payé à la souscription), et
plusieurs ensuite.
- Un paiement règle un et un seul abonnement.
```

2.2 Dictionnaire des données

```
Code               | Signification                              | Type | Taille | Remarque
num_serie          | Numéro de série de l'équipement            | AN   | 20     | Identifiant
designation        | Désignation de l'équipement                | AN   | 80     |
date_achat         | Date d'achat                               | D    |        |
etat               | État de l'équipement                       | A    | 12     | en service,
en panne, réformé
prix_achat         | Prix d'achat de l'équipement               | N    | 8,2    | >= 0
code_categorie     | Code de la catégorie d'équipement          | AN   | 10     | Identifiant
(VELO, TAPIS...)
libelle_categorie  | Libellé de la catégorie                    | AN   | 30     |
periode_jours      | Périodicité de maintenance préventive      | N    |        | en jours, > 0
siret              | Numéro SIRET du prestataire                | AN   | 14     | Identifiant,
14 chiffres
raison_sociale     | Raison sociale du prestataire              | AN   | 80     |
tel_prestataire    | Téléphone du prestataire                   | AN   | 15     |
email_prestataire  | Adresse électronique du prestataire        | AN   | 100    | facultatif
id_intervention    | Numéro de l'intervention                   | N    |        | Identifiant
(auto)
date_intervention  | Date de l'intervention                     | D    |        |
nature             | Nature de l'intervention                   | A    | 10     | préventive,
corrective
description_interv | Description de l'intervention              | AN   | 255    |
cout               | Coût de l'intervention                     | N    | 8,2    | > 0
quantite           | Quantité de matériel par participant       | N    |        | porté par
NÉCESSITER
date_demande       | Date et heure d'inscription en attente     | DH   |        | porté par
ATTENDRE
rang               | Rang dans la liste d'attente               | N    |        | porté par
ATTENDRE
id_coaching        | Numéro du coaching                         | N    |        | Identifiant
(auto)
debut_coaching     | Date et heure de début du coaching         | DH   |        |
duree              | Durée du coaching                          | N    |        | en minutes,
30 à 120
tarif              | Tarif appliqué                             | N    | 6,2    |
objectif           | Objectif de l'adhérent                     | AN   | 255    | facultatif

Question 2.2
-- Réponse : non. C'est une donnée calculée : date de la dernière maintenance préventive (ou
date
-- d'achat) + périodicité de la catégorie. Si on la stockait, il faudrait la recalculer à
chaque
-- intervention, sinon incohérence. On ne la met pas dans le dictionnaire ; on la calculera par
-- une requête / vue / fonction.
```

2.3 Dépendances fonctionnelles et règles de gestion

```
a.
num_serie       -> designation, date_achat, prix_achat, etat, code_categorie, id_salle
siret           -> raison_sociale, tel_prestataire, email_prestataire
code_categorie  -> libelle_categorie, periode_jours
```

```
id_intervention -> date_intervention, nature, description_interv, cout, num_serie, siret

b.
- La quantité ne dépend ni de l'activité seule (le body pump demande 1 barre ET 1 step) ni de
la
  catégorie seule (1 tapis pour le yoga, autre chose ailleurs) : elle dépend du couple.
  (id_activite, code_categorie) -> quantite
- Le rang dépend du couple adhérent + séance (un adhérent est en attente sur plusieurs séances,
  avec un rang différent sur chacune) :
  (id_adherent, id_seance) -> rang, date_demande
  => DF composées = associations porteuses de données.

c.
RG8  Un équipement appartient à une et une seule catégorie ; une catégorie regroupe 0 à n
équipements.
RG9  Un équipement est installé dans une et une seule salle ; une salle possède 0 à n
équipements.
RG10 Une intervention concerne un et un seul équipement ; un équipement subit 0 à n
interventions.
RG11 Une intervention est réalisée par un et un seul prestataire ; un prestataire réalise 0 à n
interventions.
RG12 Une activité nécessite 0 à n catégories d'équipement (zumba : aucune), avec une quantité
par
     participant ; une catégorie est nécessaire à 0 à n activités.
RG13 Un adhérent peut être en attente sur 0 à n séances ; une séance a 0 à n adhérents en
attente ;
     on mémorise la date de la demande et le rang.
RG14 Un coaching est suivi par un et un seul adhérent ; un adhérent suit 0 à n coachings.
RG15 Un coaching est assuré par un et un seul coach ; un coach assure 0 à n coachings.
RG16 Un coaching se déroule dans une et une seule salle ; une salle accueille 0 à n coachings.
```

2.4 MCD avec Looping

```
MCD réalisé dans Looping (Looping1.loo) :
Entités existantes (identifiant seul) : SALLE, ACTIVITÉ, COACH, ADHÉRENT, SÉANCE
Nouvelles entités : CATÉGORIE, ÉQUIPEMENT, PRESTATAIRE, INTERVENTION, COACHING
Associations :
- APPARTENIR  ÉQUIPEMENT 1,1 — 0,n CATÉGORIE
- CONTENIR    ÉQUIPEMENT 1,1 — 0,n SALLE
- CONCERNER   INTERVENTION 1,1 — 0,n ÉQUIPEMENT
- RÉALISER    INTERVENTION 1,1 — 0,n PRESTATAIRE
- NÉCESSITER  ACTIVITÉ 0,n — 0,n CATÉGORIE   (porte quantite)
- ATTENDRE    ADHÉRENT 0,n — 0,n SÉANCE      (porte date_demande, rang)
- SUIVRE      COACHING 1,1 — 0,n ADHÉRENT
- ASSURER     COACHING 1,1 — 0,n COACH
- DÉROULER    COACHING 1,1 — 0,n SALLE

Questions 2.4
1. Ternaire COACH–ADHÉRENT–SALLE : l'identifiant d'une ternaire est le triplet
   (id_coach, id_adherent, id_salle). Un même adhérent ne pourrait donc avoir qu'UN seul
coaching
   avec le même coach dans la même salle, alors que le cahier des charges dit qu'il peut en
suivre
   plusieurs, avec le même coach, éventuellement le même jour.
   -> Solution : créer une entité COACHING (id_coaching, debut, duree, tarif, objectif) reliée
par
   trois associations binaires (1,1) à ADHÉRENT, COACH et SALLE.
2. quantite se place sur l'association NÉCESSITER : elle dépend du couple (activité,
catégorie),
   pas de l'une ou l'autre seule (cf. 2.3 b).
3. Liste d'attente séparée de la réservation :
```

```
   - Pour une seule table "réservation avec statut en attente" : plus simple, passer de
l'attente
     à la réservation = un simple UPDATE du statut.
   - Contre : le rang n'a de sens que pour les attentes (colonne vide pour toutes les vraies
     réservations), et les lignes "en attente" risquent d'être comptées par erreur dans les
places
     occupées ou dans le quota hebdo de la formule. C'est pour ça qu'on la sépare.
```

2.5 Passer au MLD

```
a. (clé primaire = 1er(s) attribut(s), à souligner ; # = clé étrangère)
categorie    (code_categorie, libelle_categorie, periode_jours)
equipement   (num_serie, designation, date_achat, prix_achat, etat, #code_categorie, #id_salle)
prestataire  (siret, raison_sociale, telephone, email)
intervention (id_intervention, date_intervention, nature, description, cout, #num_serie,
#siret)
coaching     (id_coaching, debut, duree, tarif, objectif, #id_adherent, #id_coach, #id_salle)
necessiter   (#id_activite, #code_categorie, quantite)
attendre     (#id_adherent, #id_seance, date_demande, rang)

b. Règles appliquées
categorie    : R1
equipement   : R1 + R2 x2 (APPARTENIR, CONTENIR)
prestataire  : R1
intervention : R1 + R2 x2 (CONCERNER, RÉALISER)
coaching     : R1 + R2 x3 (SUIVRE, ASSURER, DÉROULER)
necessiter   : R3 (association (0,n)-(0,n) porteuse)
attendre     : R3 (association (0,n)-(0,n) porteuse)
=> 7 tables : 5 issues d'entités + 2 issues d'associations (x,n)-(x,n). OK avec le contrôle.
```

2.6 MPD

```
a/b. Script vitasportplus.sql (généré par Looping puis corrigé).
Seules les 7 nouvelles tables sont créées (salle, coach, adherent, activite, seance existent
déjà
dans vitasport), parents avant enfants.
Les 2 UNIQUE qui traduisent des règles de gestion :
- uq_coaching_coach_debut UNIQUE (id_coach, debut) : un coach ne peut pas commencer deux
coachings à la même heure
- uq_attendre_seance_rang UNIQUE (id_seance, rang) : deux adhérents ne peuvent pas avoir le
même rang sur une séance

USE vitasport;

CREATE TABLE categorie (
  code_categorie     VARCHAR(10)  PRIMARY KEY,
  libelle_categorie  VARCHAR(30)  NOT NULL UNIQUE,
  periode_jours      SMALLINT     NOT NULL,
  CONSTRAINT ck_categorie_periode CHECK (periode_jours > 0)
) ENGINE = InnoDB;

CREATE TABLE equipement (
  num_serie       VARCHAR(20)   PRIMARY KEY,
  designation     VARCHAR(80)   NOT NULL,
  date_achat      DATE          NOT NULL,
  prix_achat      DECIMAL(8,2)  NOT NULL,
  etat            VARCHAR(12)   NOT NULL DEFAULT 'en service',
  code_categorie  VARCHAR(10)   NOT NULL,
  id_salle        INT           NOT NULL,
  CONSTRAINT fk_equipement_categorie FOREIGN KEY (code_categorie)
```

```
    REFERENCES categorie(code_categorie),
  CONSTRAINT fk_equipement_salle     FOREIGN KEY (id_salle)
    REFERENCES salle(id_salle),
  CONSTRAINT ck_equipement_etat CHECK (etat IN ('en service','en panne','réformé')),
  CONSTRAINT ck_equipement_prix CHECK (prix_achat >= 0)
) ENGINE = InnoDB;

CREATE TABLE prestataire (
  siret           CHAR(14)      PRIMARY KEY,
  raison_sociale  VARCHAR(80)   NOT NULL,
  telephone       VARCHAR(15)   NOT NULL,
  email           VARCHAR(100)  NULL,
  CONSTRAINT ck_prestataire_siret CHECK (siret REGEXP '^[0-9]{14}$')
) ENGINE = InnoDB;

CREATE TABLE intervention (
  id_intervention    INT           AUTO_INCREMENT PRIMARY KEY,
  date_intervention  DATE          NOT NULL,
  nature             VARCHAR(10)   NOT NULL,
  description        VARCHAR(255)  NOT NULL,
  cout               DECIMAL(8,2)  NOT NULL,
  num_serie          VARCHAR(20)   NOT NULL,
  siret              CHAR(14)      NOT NULL,
  CONSTRAINT fk_intervention_equipement  FOREIGN KEY (num_serie)
    REFERENCES equipement(num_serie),
  CONSTRAINT fk_intervention_prestataire FOREIGN KEY (siret)
    REFERENCES prestataire(siret),
  CONSTRAINT ck_intervention_nature CHECK (nature IN ('preventive','corrective')),
  CONSTRAINT ck_intervention_cout   CHECK (cout > 0)
) ENGINE = InnoDB;

CREATE TABLE coaching (
  id_coaching  INT           AUTO_INCREMENT PRIMARY KEY,
  debut        DATETIME      NOT NULL,
  duree        SMALLINT      NOT NULL,
  tarif        DECIMAL(6,2)  NOT NULL,
  objectif     VARCHAR(255)  NULL,
  id_adherent  INT           NOT NULL,
  id_coach     INT           NOT NULL,
  id_salle     INT           NOT NULL,
  CONSTRAINT fk_coaching_adherent FOREIGN KEY (id_adherent)
    REFERENCES adherent(id_adherent) ON DELETE CASCADE,
  CONSTRAINT fk_coaching_coach    FOREIGN KEY (id_coach)
    REFERENCES coach(id_coach),
  CONSTRAINT fk_coaching_salle    FOREIGN KEY (id_salle)
    REFERENCES salle(id_salle),
  CONSTRAINT uq_coaching_coach_debut UNIQUE (id_coach, debut),
  CONSTRAINT ck_coaching_duree CHECK (duree BETWEEN 30 AND 120),
  CONSTRAINT ck_coaching_tarif CHECK (tarif >= 0)
) ENGINE = InnoDB;

CREATE TABLE necessiter (
  id_activite     INT          NOT NULL,
  code_categorie  VARCHAR(10)  NOT NULL,
  quantite        TINYINT      NOT NULL DEFAULT 1,
  PRIMARY KEY (id_activite, code_categorie),
  CONSTRAINT fk_necessiter_activite  FOREIGN KEY (id_activite)
    REFERENCES activite(id_activite) ON DELETE CASCADE,
  CONSTRAINT fk_necessiter_categorie FOREIGN KEY (code_categorie)
    REFERENCES categorie(code_categorie) ON DELETE CASCADE,
  CONSTRAINT ck_necessiter_quantite CHECK (quantite > 0)
) ENGINE = InnoDB;
```

```
CREATE TABLE attendre (
  id_adherent   INT       NOT NULL,
  id_seance     INT       NOT NULL,
  date_demande  DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
  rang          TINYINT   NOT NULL,
  PRIMARY KEY (id_adherent, id_seance),
  CONSTRAINT fk_attendre_adherent FOREIGN KEY (id_adherent)
    REFERENCES adherent(id_adherent) ON DELETE CASCADE,
  CONSTRAINT fk_attendre_seance   FOREIGN KEY (id_seance)
    REFERENCES seance(id_seance) ON DELETE CASCADE,
  CONSTRAINT uq_attendre_seance_rang UNIQUE (id_seance, rang),
  CONSTRAINT ck_attendre_rang CHECK (rang > 0)
) ENGINE = InnoDB;

c. Jeu d'essai (après avoir chargé vitasport_schema.sql + vitasport_data.sql, partie 4)
USE vitasport;

INSERT INTO categorie (code_categorie, libelle_categorie, periode_jours) VALUES
  ('VELO',  'Vélo de cycling', 90),
  ('TAPIS', 'Tapis de sol',    180);

INSERT INTO equipement (num_serie, designation, date_achat, prix_achat, code_categorie,
id_salle) VALUES
  ('VC-2023-002', 'Vélo Keiser M3i',    '2023-03-15', 1890.00, 'VELO',  1),
  ('VC-2024-017', 'Vélo Schwinn AC',    '2024-01-10', 1650.00, 'VELO',  2),
  ('TP-2024-101', 'Lot de 20 tapis',    '2024-06-01',  400.00, 'TAPIS', 1);
-- etat non renseigné -> DEFAULT 'en service'

INSERT INTO prestataire (siret, raison_sociale, telephone, email) VALUES
  ('51234567800025', 'Cardio Répar', '0472000000', NULL);

INSERT INTO intervention (date_intervention, nature, description, cout, num_serie, siret)
VALUES
  ('2026-09-28', 'corrective', 'Remplacement courroie et vis de carter', 89.70, 'VC-2023-002',
'51234567800025'),
  ('2026-10-02', 'preventive', 'Révision trimestrielle',                 60.00, 'VC-2024-017',
'51234567800025');

INSERT INTO necessiter (id_activite, code_categorie, quantite) VALUES
  (5, 'VELO', 1);                 -- Cycling : 1 vélo par participant

INSERT INTO attendre (id_adherent, id_seance, rang) VALUES
  (1, 1, 1);                      -- date_demande -> DEFAULT CURRENT_TIMESTAMP

INSERT INTO coaching (debut, duree, tarif, objectif, id_adherent, id_coach, id_salle) VALUES
  ('2026-10-14 18:00:00', 60, 45.00, 'Reprise après blessure', 2, 1, 1);

d. Tester chaque contrainte (chaque INSERT/UPDATE doit être REFUSÉ)
-- CHECK etat
UPDATE equipement SET etat = 'cassé' WHERE num_serie = 'VC-2023-002';           -- 3819
ck_equipement_etat
-- CHECK prix d'achat
INSERT INTO equipement VALUES ('X1','Test','2026-01-01',-10,'en service','VELO',1); -- 3819
ck_equipement_prix
-- CHECK nature
INSERT INTO intervention (date_intervention, nature, description, cout, num_serie, siret)
VALUES ('2026-10-12','urgente','test',50,'VC-2023-002','51234567800025');        -- 3819
ck_intervention_nature
-- CHECK cout positif
INSERT INTO intervention (date_intervention, nature, description, cout, num_serie, siret)
VALUES ('2026-10-12','corrective','test',0,'VC-2023-002','51234567800025');      -- 3819
```

```
ck_intervention_cout
-- CHECK siret 14 chiffres
INSERT INTO prestataire VALUES ('123', 'Test', '0600000000', NULL);              -- 3819
ck_prestataire_siret
-- CHECK periode > 0
INSERT INTO categorie VALUES ('STEP', 'Step', 0);                                -- 3819
ck_categorie_periode
-- CHECK duree coaching 30-120
INSERT INTO coaching (debut, duree, tarif, id_adherent, id_coach, id_salle)
VALUES ('2026-10-15 10:00:00', 150, 40, 3, 2, 1);                                -- 3819
ck_coaching_duree
-- UNIQUE coach + debut (même coach, même heure que le coaching déjà inséré)
INSERT INTO coaching (debut, duree, tarif, id_adherent, id_coach, id_salle)
VALUES ('2026-10-14 18:00:00', 45, 40, 3, 1, 2);                                -- 1062
uq_coaching_coach_debut
-- UNIQUE seance + rang (rang 1 déjà pris sur la séance 1)
INSERT INTO attendre (id_adherent, id_seance, rang) VALUES (3, 1, 1);           -- 1062
uq_attendre_seance_rang
-- CHECK rang > 0
INSERT INTO attendre (id_adherent, id_seance, rang) VALUES (4, 1, 0);           -- 3819
ck_attendre_rang
-- CHECK quantite > 0
INSERT INTO necessiter VALUES (3, 'TAPIS', 0);                                   -- 3819
ck_necessiter_quantite
-- FK : catégorie inexistante
INSERT INTO equipement VALUES ('X2','Test','2026-01-01',10,'en service','RAMEUR',1); -- 1452
fk_equipement_categorie
-- FK RESTRICT : supprimer un équipement qui a des interventions
DELETE FROM equipement WHERE num_serie = 'VC-2023-002';                          -- 1451
fk_intervention_equipement
-- ON DELETE CASCADE (pas une erreur, à observer) : supprimer l'adhérent 2 supprime aussi son
coaching
-- (à faire dans une transaction + ROLLBACK pour ne pas abîmer la base)
```

2.7 Normalisation

```
a. Clé primaire du tableau : (n_interv, ref_piece)
-- n_interv seul se répète (l'intervention 3 utilise 2 pièces), ref_piece seul aussi (CR-88
utilisée
-- dans 2 interventions). Le couple est unique : une ligne = une pièce utilisée dans une
intervention.
b. Dépendances fonctionnelles :
n_interv              -> date, num_serie, siret
num_serie             -> designation, salle
siret                 -> prestataire
ref_piece             -> libelle_piece, prix_piece
(n_interv, ref_piece) -> qte
-- designation et salle dépendent de n_interv seulement EN PASSANT PAR num_serie
-- (n_interv -> num_serie -> salle) : ce sont des DF transitives.
-- qte dépend du couple : la quantité d'UNE pièce dans UNE intervention.
c. Forme normale : 1FN seulement.
-- 1FN : il y a une clé et chaque case contient une seule valeur.
-- Pas en 2FN : des colonnes dépendent d'une PARTIE de la clé seulement
-- (n_interv -> date ; ref_piece -> libelle_piece, prix_piece). Seule qte dépend de toute la
clé.
Anomalies :
-- Mise à jour : le prix de la courroie CR-88 est recopié sur 2 lignes. S'il passe à 42 €, il
faut
--   modifier toutes les lignes ; si on en oublie une, la même pièce a deux prix ->
incohérence.
```

```
-- Insertion : impossible d'enregistrer une nouvelle pièce du catalogue tant qu'aucune
intervention
--   ne l'a utilisée, car n_interv fait partie de la clé primaire et ne peut pas être NULL.
-- Suppression : si on supprime l'intervention 6, on perd aussi la seule trace du vélo VC-2024-
017
--   (Vélo Schwinn AC, salle Gratte-Ciel), alors qu'il existe toujours.
d. Décomposition en 3FN (MLD textuel) :
prestataire  (siret, raison_sociale)
equipement   (num_serie, designation, #id_salle)
intervention (n_interv, date_interv, #num_serie, #siret)
piece        (ref_piece, libelle_piece, prix_piece)
utiliser     (#n_interv, #ref_piece, qte)
-- Méthode : chaque partie gauche de DF devient une table (= sa clé primaire) et emporte ses
colonnes.
-- salle devient la clé étrangère #id_salle vers la table salle existante.
-- Toutes les tables sont en 3FN : chaque colonne dépend de la clé, de toute la clé, et de rien
d'autre.

Lien avec le MCD du 2.4 :
-- PRESTATAIRE, ÉQUIPEMENT et INTERVENTION existent déjà dans le MCD (mêmes identifiants :
siret,
--   num_serie, id_intervention) : la normalisation retrouve le même découpage.
-- Elle fait apparaître une entité NOUVELLE, absente du cahier des charges : PIÈCE (ref_piece,
--   libelle_piece, prix_piece), reliée à INTERVENTION par une association UTILISER (0,n)-(0,n)
--   qui porte la propriété qte (DF composée -> association porteuse, devient une table par
R3).
-- Remarque : si le prix d'une pièce peut changer dans le temps, il faut mémoriser le prix
facturé
--   dans UTILISER (il dépend alors du couple intervention + pièce).
```

PARTIE 3 — UML avec draw.io

3.1 Diagramme de classes (classes.drawio)

```
Classes existantes (2 attributs) :
Salle     : - idSalle : int ; - nom : String
Activite  : - idActivite : int ; - libelle : String
Coach     : - idCoach : int ; - nom : String
Adherent  : - idAdherent : int ; - nom : String
Seance    : - idSeance : int ; - debut : DateTime

Nouvelles classes :
CategorieEquipement : - code : String ; - libelle : String ; - periodeJours : int
                      + calculerProchaineMaintenance(e : Equipement) : Date
Equipement   : - numSerie : String ; - designation : String ; - dateAchat : Date ;
               - prixAchat : float ; - etat : String
               + declarerPanne() : void ; + reformer() : void
Prestataire  : - siret : String ; - raisonSociale : String ; - telephone : String ; - email :
String
Intervention : - idIntervention : int ; - dateIntervention : Date ; - nature : String ;
               - description : String ; - cout : float
Coaching     : - idCoaching : int ; - debut : DateTime ; - duree : int ; - tarif : float ; -
objectif : String

Classes-associations (trait pointillé vers le milieu de l'association) :
Necessiter : - quantite : int                          (entre Activite et CategorieEquipement)
Attente    : - dateDemande : DateTime ; - rang : int   (entre Adherent et Seance)

Multiplicités :
Equipement   *  ——  1 CategorieEquipement
```

```
Equipement   *  ——  1 Salle
Intervention *  ——  1 Equipement
Intervention *  ——  1 Prestataire
Coaching     *  ——  1 Adherent
Coaching     *  ——  1 Coach
Coaching     *  ——  1 Salle
Activite     *  ——  * CategorieEquipement   (+ classe-association Necessiter)
Adherent     *  ——  * Seance                (+ classe-association Attente)
-- Conversion : 1,1 -> 1 ; 0,1 -> 0..1 ; 0,n -> * ; 1,n -> 1..*  ... placé de l'AUTRE côté.

Question 3.1
-- Merise CONCERNER : INTERVENTION (1,1) — (0,n) ÉQUIPEMENT
-- UML : Intervention  *  ——  1  Equipement
-- Réponse : en Merise, la cardinalité est écrite à côté de l'entité qu'on interroge (« une
intervention
-- concerne combien d'équipements ? » -> 1,1 côté INTERVENTION). En UML, la multiplicité est
écrite à
-- côté de la classe dont on compte les objets (« une intervention est liée à 1 équipement » ->
1 côté
-- Equipement). La même information se retrouve donc de l'autre côté du trait : les
cardinalités
-- « s'inversent ».
```

3.2 Diagramme de cas d'utilisation (cas_utilisation.drawio)

```
Système : « Module maintenance et coaching »
Acteurs : Coach, Responsable de salle, Prestataire, Adhérent (à gauche) ; Service de paiement
(à droite, système externe)
Cas d'utilisation :
- Déclarer une panne                  <- Coach, Responsable de salle
- Planifier une intervention          <- Responsable de salle
- Réformer un équipement              <- Responsable de salle
- Consulter l'historique des interventions
- Saisir le compte rendu d'intervention <- Prestataire
- Gérer ses disponibilités            <- Coach
- Réserver un coaching                <- Adhérent
- Payer en ligne                      <- (Adhérent), Service de paiement
- S'authentifier
Relations :
- Planifier une intervention  --«include»-->  Consulter l'historique
- Réformer un équipement      --«include»-->  Consulter l'historique
- Payer en ligne              --«extend»-->   Réserver un coaching   (condition : coaching non
inclus dans la formule)
- Réserver un coaching        --«include»-->  S'authentifier
- Gérer ses disponibilités    --«include»-->  S'authentifier

Questions 3.2
1. Réserver un coaching / Payer en ligne : «extend», car le paiement n'a lieu que dans certains
cas
   (coaching non inclus dans la formule). La flèche part de « Payer en ligne » (l'extension) et
pointe
   vers « Réserver un coaching » (le cas de base).
   Planifier une intervention / Consulter l'historique : «include», car la consultation est
   OBLIGATOIRE à chaque planification. La flèche part de « Planifier une intervention » et
pointe vers
   « Consulter l'historique ».
2. Oui, le prestataire est un acteur : c'est une entité EXTÉRIEURE au système qui interagit
   directement avec lui (il saisit lui-même en ligne le compte rendu de son intervention).
3. Bonus : le diagramme de séquence (échanges dans l'ordre chronologique entre l'adhérent,
   l'application et le service de paiement).
```

PARTIE 4 — Charger la base de référence

4.1

```
Exécuter dans l'ordre : vitasport_schema.sql, vitasport_data.sql, puis vitasportplus.sql
(vitasport_schema.sql supprime la base vitasport : les tables de la partie 2 sont à recréer).
```

4.2

```
SELECT table_name, table_rows
FROM information_schema.tables
WHERE table_schema = 'vitasport'
ORDER BY table_name;
-- 17 tables (10 de référence + 7 de VitaSport+). table_rows n'est qu'une estimation en InnoDB,
-- les comptes exacts se vérifient avec COUNT(*) :
SELECT 'salle' AS t, COUNT(*) FROM salle
UNION ALL SELECT 'activite', COUNT(*) FROM activite
UNION ALL SELECT 'coach', COUNT(*) FROM coach
UNION ALL SELECT 'coach_activite', COUNT(*) FROM coach_activite
UNION ALL SELECT 'formule', COUNT(*) FROM formule
UNION ALL SELECT 'adherent', COUNT(*) FROM adherent
UNION ALL SELECT 'abonnement', COUNT(*) FROM abonnement
UNION ALL SELECT 'paiement', COUNT(*) FROM paiement
UNION ALL SELECT 'seance', COUNT(*) FROM seance
UNION ALL SELECT 'reservation', COUNT(*) FROM reservation;
-- Attendu : salle 3, activite 9, coach 6, coach_activite 16, formule 6, adherent 40,
-- abonnement 60, paiement 509, seance 66, reservation 255.
```

4.3

```
SHOW CREATE TABLE seance;
-- Contraintes nommées : fk_seance_activite, fk_seance_coach, fk_seance_salle (clés
étrangères),
-- uq_seance_coach_debut (UNIQUE : un coach ne donne qu'un cours à la fois),
-- ck_seance_places (CHECK : places_max entre 1 et 40).
```

PARTIE 5 — SQL avancé (date du jour : 12/10/2026)

A. Jointures

```
-- Q1 : planning du 12/10 (6 lignes)
SELECT DATE_FORMAT(s.debut, '%H:%i') AS heure, sa.nom AS salle, a.libelle AS activite,
       CONCAT(c.prenom, ' ', c.nom) AS coach, s.places_max AS places
FROM seance s
JOIN salle sa   ON sa.id_salle   = s.id_salle
JOIN activite a ON a.id_activite = s.id_activite
JOIN coach c    ON c.id_coach    = s.id_coach
WHERE s.debut >= '2026-10-12' AND s.debut < '2026-10-13'
ORDER BY heure, salle;
-- Pas de DATE(s.debut) = ... : une fonction sur la colonne empêche l'utilisation de l'index.
-- On encadre la journée : >= le 12 à 00:00 et < le 13 à 00:00.

-- Q2 : adhérents avec un abonnement valide (38 lignes)
SELECT ad.nom, ad.prenom, sa.nom AS salle_principale, f.libelle AS formule, ab.date_fin
FROM adherent ad
JOIN salle sa      ON sa.id_salle    = ad.id_salle_principale
JOIN abonnement ab ON ab.id_adherent = ad.id_adherent
```

```
JOIN formule f     ON f.id_formule   = ab.id_formule
WHERE '2026-10-12' BETWEEN ab.date_debut AND ab.date_fin
ORDER BY salle_principale, ad.nom;

-- Q3 : nombre de séances par coach, y compris 0 (6 lignes, Hugo Lambert à 0)
SELECT c.id_coach, CONCAT(c.prenom, ' ', c.nom) AS coach, COUNT(s.id_seance) AS nb_seances
FROM coach c
LEFT JOIN seance s ON s.id_coach = c.id_coach
GROUP BY c.id_coach, c.prenom, c.nom
ORDER BY nb_seances DESC;
-- LEFT JOIN pour garder les coachs sans séance. COUNT(s.id_seance) et pas COUNT(*) :
-- COUNT(*) compterait 1 pour la ligne "vide" du coach sans séance.

-- Q4 : adhérents sans abonnement valide (2 lignes : Éric Nicolas, Clara Henry)
-- Version fausse (condition de date dans le WHERE) -> 0 ligne :
SELECT ad.id_adherent, ad.prenom, ad.nom
FROM adherent ad
LEFT JOIN abonnement ab ON ab.id_adherent = ad.id_adherent
WHERE ab.id_abonnement IS NULL
  AND '2026-10-12' BETWEEN ab.date_debut AND ab.date_fin;
-- Version juste (condition de date dans le ON) :
SELECT ad.id_adherent, ad.prenom, ad.nom
FROM adherent ad
LEFT JOIN abonnement ab ON ab.id_adherent = ad.id_adherent
                       AND '2026-10-12' BETWEEN ab.date_debut AND ab.date_fin
WHERE ab.id_abonnement IS NULL;
-- Réponse : le WHERE s'applique APRÈS la jointure. Les lignes qu'on cherche ont ab.* à NULL,
-- donc "BETWEEN ab.date_debut AND ab.date_fin" vaut NULL (ni vrai ni faux) et les élimine
toutes.
-- Dans le ON, la date fait partie de la condition de jointure : on ne joint que les
abonnements
-- valides, et les adhérents qui n'en ont aucun ressortent avec ab.* à NULL.

-- Q5 : formules jamais souscrites (1 ligne : Entreprise)
SELECT f.libelle
FROM abonnement ab
RIGHT JOIN formule f ON f.id_formule = ab.id_formule
WHERE ab.id_abonnement IS NULL;

SELECT f.libelle
FROM formule f
LEFT JOIN abonnement ab ON ab.id_formule = f.id_formule
WHERE ab.id_abonnement IS NULL;
-- Même résultat : A RIGHT JOIN B = B LEFT JOIN A. On garde toutes les lignes de la table
"conservée".

-- Q6 : parrains, nombre de filleuls et prénoms (4 lignes, Louise Robert a 2 filleuls)
SELECT p.id_adherent, CONCAT(p.prenom, ' ', p.nom) AS parrain,
       COUNT(fi.id_adherent) AS nb_filleuls,
       GROUP_CONCAT(fi.prenom ORDER BY fi.prenom SEPARATOR ', ') AS filleuls
FROM adherent p
JOIN adherent fi ON fi.id_parrain = p.id_adherent
GROUP BY p.id_adherent, p.prenom, p.nom
ORDER BY nb_filleuls DESC;
-- Auto-jointure : la même table deux fois avec deux alias (p = parrain, fi = filleul).

-- Q7 : chaque couple coach x activité, oui/non (54 lignes = 6 coachs x 9 activités)
SELECT c.nom AS coach, a.libelle AS activite,
       CASE WHEN ca.id_coach IS NULL THEN 'non' ELSE 'oui' END AS habilite
FROM coach c
CROSS JOIN activite a
LEFT JOIN coach_activite ca ON ca.id_coach = c.id_coach AND ca.id_activite = a.id_activite
```

```
ORDER BY c.nom, a.libelle;
-- CROSS JOIN = toutes les combinaisons ; le LEFT JOIN regarde si la combinaison existe dans
coach_activite.

-- Q8 : chaque coach et ses activités sur une ligne (6 lignes)
SELECT CONCAT(c.prenom, ' ', c.nom) AS coach,
       GROUP_CONCAT(a.libelle ORDER BY a.libelle SEPARATOR ', ') AS activites
FROM coach c
LEFT JOIN coach_activite ca ON ca.id_coach    = c.id_coach
LEFT JOIN activite a        ON a.id_activite = ca.id_activite
GROUP BY c.id_coach, c.prenom, c.nom;

-- Q9 : séances animées par un coach non habilité (0 ligne)
SELECT s.id_seance, s.debut, c.nom AS coach, a.libelle AS activite
FROM seance s
JOIN coach c    ON c.id_coach    = s.id_coach
JOIN activite a ON a.id_activite = s.id_activite
LEFT JOIN coach_activite ca ON ca.id_coach = s.id_coach AND ca.id_activite = s.id_activite
WHERE ca.id_coach IS NULL;
```

B. Agrégation

```
-- Q10 : séances non annulées par salle (3 lignes : Gratte-Ciel 21, les autres 22)
SELECT sa.nom AS salle, COUNT(*) AS nb_seances
FROM seance s
JOIN salle sa ON sa.id_salle = s.id_salle
WHERE s.annulee = FALSE
GROUP BY sa.id_salle, sa.nom;

-- Q11 : les 10 séances les plus remplies du 12 au 17/10 (la 55 est à 100 %)
SELECT s.id_seance, DATE_FORMAT(s.debut, '%d/%m %H:%i') AS jour_heure, a.libelle AS activite,
       s.places_max AS places, COUNT(r.id_adherent) AS nb_inscrits,
       ROUND(100 * COUNT(r.id_adherent) / s.places_max, 1) AS taux_pct
FROM seance s
JOIN activite a ON a.id_activite = s.id_activite
LEFT JOIN reservation r ON r.id_seance = s.id_seance AND r.statut <> 'annulee'
WHERE s.annulee = FALSE
  AND s.debut >= '2026-10-12' AND s.debut < '2026-10-18'
GROUP BY s.id_seance, s.debut, a.libelle, s.places_max
ORDER BY taux_pct DESC
LIMIT 10;

-- Q12 : CA et nombre de paiements par mois en 2026 (10 lignes)
SELECT DATE_FORMAT(date_paiement, '%Y-%m') AS mois, SUM(montant) AS ca, COUNT(*) AS
nb_paiements
FROM paiement
WHERE date_paiement >= '2026-01-01' AND date_paiement < '2027-01-01'
GROUP BY mois
ORDER BY mois;

-- Q13 : CA 2026 par salle principale et mode de paiement, avec sous-totaux (13 lignes)
SELECT sa.nom AS salle, p.mode, SUM(p.montant) AS ca
FROM paiement p
JOIN abonnement ab ON ab.id_abonnement = p.id_abonnement
JOIN adherent ad   ON ad.id_adherent   = ab.id_adherent
JOIN salle sa      ON sa.id_salle      = ad.id_salle_principale
WHERE p.date_paiement >= '2026-01-01' AND p.date_paiement < '2027-01-01'
GROUP BY sa.nom, p.mode WITH ROLLUP;
-- Réponse : les lignes de sous-total ont NULL dans la colonne regroupée. mode = NULL -> sous-
total
-- de la salle ; salle = NULL et mode = NULL -> total général (9 541,20 €).
```

```sql
-- Q14 : présents, absents et taux de présence par coach (5 lignes)
SELECT CONCAT(c.prenom, ' ', c.nom) AS coach,
       SUM(r.statut = 'presente') AS presents,
       SUM(r.statut = 'absente')  AS absents,
       ROUND(100 * SUM(r.statut = 'presente')
             / (SUM(r.statut = 'presente') + SUM(r.statut = 'absente')), 1) AS taux_presence
FROM coach c
JOIN seance s      ON s.id_coach  = c.id_coach
JOIN reservation r ON r.id_seance = s.id_seance
GROUP BY c.id_coach, c.prenom, c.nom
ORDER BY taux_presence DESC;
-- Astuce : (r.statut = 'presente') vaut 1 si vrai, 0 si faux -> SUM compte les présents.

-- Q15 : formules en cours, nombre d'adhérents et âge moyen (5 lignes)
SELECT f.libelle, COUNT(DISTINCT ab.id_adherent) AS nb_adherents,
       ROUND(AVG(TIMESTAMPDIFF(YEAR, ad.date_naissance, '2026-10-12')), 1) AS age_moyen
FROM formule f
JOIN abonnement ab ON ab.id_formule  = f.id_formule
JOIN adherent ad   ON ad.id_adherent = ab.id_adherent
WHERE '2026-10-12' BETWEEN ab.date_debut AND ab.date_fin
GROUP BY f.id_formule, f.libelle;

-- Q16 : réservations non annulées par jour de la semaine (6 lignes)
SELECT ELT(DAYOFWEEK(s.debut), 'dimanche', 'lundi', 'mardi', 'mercredi',
           'jeudi', 'vendredi', 'samedi') AS jour,
       COUNT(*) AS nb_resa
FROM reservation r
JOIN seance s ON s.id_seance = r.id_seance
WHERE r.statut <> 'annulee'
GROUP BY DAYOFWEEK(s.debut)
ORDER BY nb_resa DESC;
-- DAYOFWEEK renvoie 1 (dimanche) à 7 (samedi) ; ELT(n, ...) renvoie le n-ième texte de la
liste.

Question B
-- Réponse : on veut garder les séances qui n'ont aucune réservation (0 inscrit). Avec
-- r.statut <> 'annulee' dans le WHERE, ces séances (r.statut à NULL) seraient éliminées et le
-- LEFT JOIN deviendrait un INNER JOIN. Dans le ON, on filtre seulement les réservations
jointes.
-- COUNT(r.id_adherent) ne compte que les valeurs non NULL : une séance sans réservation donne
0.
-- COUNT(*) compterait la ligne de la séance elle-même et donnerait 1 inscrit au lieu de 0.
```

C. Sous-requêtes

```sql
-- Q17 : séances avec plus de places que la moyenne (24 lignes)
SELECT id_seance, debut, places_max
FROM seance
WHERE places_max > (SELECT AVG(places_max) FROM seance);

-- Q18 : adhérents ayant réservé au moins une séance d'intensité 3 (32 lignes)
SELECT id_adherent, prenom, nom
FROM adherent
WHERE id_adherent IN (SELECT r.id_adherent
                      FROM reservation r
                      JOIN seance s   ON s.id_seance   = r.id_seance
                      JOIN activite a ON a.id_activite = s.id_activite
                      WHERE a.intensite = 3);

-- Q19 : abonnement valide et jamais de séance d'intensité 3 (6 lignes)
```

```sql
SELECT ad.id_adherent, ad.prenom, ad.nom
FROM adherent ad
WHERE EXISTS (SELECT 1 FROM abonnement ab
              WHERE ab.id_adherent = ad.id_adherent
                AND '2026-10-12' BETWEEN ab.date_debut AND ab.date_fin)
  AND NOT EXISTS (SELECT 1
                  FROM reservation r
                  JOIN seance s   ON s.id_seance   = r.id_seance
                  JOIN activite a ON a.id_activite = s.id_activite
                  WHERE r.id_adherent = ad.id_adherent AND a.intensite = 3);

-- Q20 : adhérents qui ne sont parrains de personne
-- Version de l'énoncé -> 0 ligne :
SELECT * FROM adherent WHERE id_adherent NOT IN (SELECT id_parrain FROM adherent);
-- Version corrigée -> 36 lignes :
SELECT * FROM adherent
WHERE id_adherent NOT IN (SELECT id_parrain FROM adherent WHERE id_parrain IS NOT NULL);
-- Réponse : la sous-requête renvoie aussi des NULL (adhérents sans parrain). "x NOT IN (1, 5,
NULL)"
-- revient à "x <> 1 AND x <> 5 AND x <> NULL" ; or "x <> NULL" vaut NULL (inconnu), donc toute
la
-- condition n'est jamais vraie -> 0 ligne. Il faut exclure les NULL (ou utiliser NOT EXISTS).

-- Q21 : pour chaque activité, la ou les séances les plus réservées (13 lignes)
SELECT a.libelle, s.id_seance, COUNT(r.id_adherent) AS nb_resa
FROM seance s
JOIN activite a ON a.id_activite = s.id_activite
LEFT JOIN reservation r ON r.id_seance = s.id_seance AND r.statut <> 'annulee'
GROUP BY s.id_seance, a.libelle, s.id_activite
HAVING COUNT(r.id_adherent) >= ALL (
    SELECT COUNT(r2.id_adherent)
    FROM seance s2
    LEFT JOIN reservation r2 ON r2.id_seance = s2.id_seance AND r2.statut <> 'annulee'
    WHERE s2.id_activite = s.id_activite          -- corrélation avec la requête principale
    GROUP BY s2.id_seance)
ORDER BY a.libelle;
-- Plusieurs lignes par activité quand il y a égalité (ex. Yoga : séances 22 et 55 à 12).

-- Q22 : salles dont le CA 2026 dépasse la moyenne (2 lignes : Part-Dieu 3 818,40 ; Gratte-Ciel
3 354,80)
SELECT ca_salle.salle, ca_salle.ca
FROM (SELECT sa.nom AS salle, SUM(p.montant) AS ca
      FROM paiement p
      JOIN abonnement ab ON ab.id_abonnement = p.id_abonnement
      JOIN adherent ad   ON ad.id_adherent   = ab.id_adherent
      JOIN salle sa      ON sa.id_salle      = ad.id_salle_principale
      WHERE p.date_paiement >= '2026-01-01' AND p.date_paiement < '2027-01-01'
      GROUP BY sa.id_salle, sa.nom) AS ca_salle
WHERE ca_salle.ca > (SELECT AVG(ca)
                     FROM (SELECT SUM(p.montant) AS ca
                           FROM paiement p
                           JOIN abonnement ab ON ab.id_abonnement = p.id_abonnement
                           JOIN adherent ad   ON ad.id_adherent   = ab.id_adherent
                           WHERE p.date_paiement >= '2026-01-01' AND p.date_paiement < '2027-
01-01'
                           GROUP BY ad.id_salle_principale) AS t);

-- Q23 : même chose avec WITH (2 lignes)
WITH ca_salle AS (
    SELECT sa.nom AS salle, SUM(p.montant) AS ca
    FROM paiement p
    JOIN abonnement ab ON ab.id_abonnement = p.id_abonnement
```

```
    JOIN adherent ad   ON ad.id_adherent   = ab.id_adherent
    JOIN salle sa      ON sa.id_salle      = ad.id_salle_principale
    WHERE p.date_paiement >= '2026-01-01' AND p.date_paiement < '2027-01-01'
    GROUP BY sa.id_salle, sa.nom
)
SELECT salle, ca
FROM ca_salle
WHERE ca > (SELECT AVG(ca) FROM ca_salle);
-- Réponse : la version WITH est plus lisible : le calcul du CA par salle est écrit une seule
fois,
-- nommé, puis réutilisé ; dans Q22 il est recopié deux fois.

-- Q24 : la ou les formules les plus souscrites (1 ligne : Essentiel, 20)
SELECT f.libelle, COUNT(*) AS nb_abonnements
FROM abonnement ab
JOIN formule f ON f.id_formule = ab.id_formule
GROUP BY f.id_formule, f.libelle
HAVING COUNT(*) >= ALL (SELECT COUNT(*) FROM abonnement GROUP BY id_formule);

-- Q25 : adhérents ayant réservé dans une autre salle que leur salle principale (6 lignes)
SELECT DISTINCT ad.id_adherent, ad.prenom, ad.nom, sp.nom AS salle_principale
FROM adherent ad
JOIN salle sp      ON sp.id_salle     = ad.id_salle_principale
JOIN reservation r ON r.id_adherent   = ad.id_adherent
JOIN seance s      ON s.id_seance     = r.id_seance
WHERE s.id_salle <> ad.id_salle_principale;
```

D. Vues

```
-- V1 : v_planning
CREATE OR REPLACE VIEW v_planning AS
SELECT s.id_seance, s.debut, sa.nom AS salle, a.libelle AS activite,
       CONCAT(c.prenom, ' ', c.nom) AS coach, s.places_max,
       COUNT(r.id_adherent)                AS nb_inscrits,
       s.places_max - COUNT(r.id_adherent) AS places_restantes
FROM seance s
JOIN salle sa   ON sa.id_salle   = s.id_salle      -- INNER : toute séance a une salle
JOIN activite a ON a.id_activite = s.id_activite   -- INNER : toute séance a une activité
JOIN coach c    ON c.id_coach    = s.id_coach      -- INNER : toute séance a un coach
LEFT JOIN reservation r ON r.id_seance = s.id_seance
                       AND r.statut <> 'annulee'   -- LEFT : garder les séances sans
réservation
WHERE s.annulee = FALSE
GROUP BY s.id_seance, s.debut, sa.nom, a.libelle, c.prenom, c.nom, s.places_max;

-- V2 : séances du mercredi 14/10 avec au moins 5 places restantes (6 lignes)
SELECT *
FROM v_planning
WHERE debut >= '2026-10-14' AND debut < '2026-10-15'
  AND places_restantes >= 5;

-- V3 : v_ca_mensuel + CA du 3e trimestre 2026 par salle (3 lignes, Part-Dieu en tête)
CREATE OR REPLACE VIEW v_ca_mensuel AS
SELECT DATE_FORMAT(p.date_paiement, '%Y-%m') AS mois, sa.nom AS salle, SUM(p.montant) AS ca
FROM paiement p
JOIN abonnement ab ON ab.id_abonnement = p.id_abonnement
JOIN adherent ad   ON ad.id_adherent   = ab.id_adherent
JOIN salle sa      ON sa.id_salle      = ad.id_salle_principale
GROUP BY mois, sa.nom;

SELECT salle, SUM(ca) AS ca_t3
```

```
FROM v_ca_mensuel
WHERE mois BETWEEN '2026-07' AND '2026-09'
GROUP BY salle
ORDER BY ca_t3 DESC;

-- V4 : v_adherent_public + adhérents de Vénissieux (10 lignes)
CREATE OR REPLACE VIEW v_adherent_public AS
SELECT ad.id_adherent, ad.prenom, CONCAT(LEFT(ad.nom, 1), '.') AS initiale, sa.nom AS salle
FROM adherent ad
JOIN salle sa ON sa.id_salle = ad.id_salle_principale;

SELECT * FROM v_adherent_public WHERE salle = 'VitaSport Vénissieux';

-- V5 : v_seance_partdieu avec WITH CHECK OPTION
CREATE OR REPLACE VIEW v_seance_partdieu AS
SELECT id_seance, id_activite, id_coach, id_salle, debut, places_max
FROM seance
WHERE id_salle = 1
WITH CHECK OPTION;

INSERT INTO v_seance_partdieu (id_activite, id_coach, id_salle, debut, places_max)
VALUES (1, 1, 2, '2026-10-19 12:15:00', 12);   -- REFUSÉ : erreur 1369 CHECK OPTION failed
INSERT INTO v_seance_partdieu (id_activite, id_coach, id_salle, debut, places_max)
VALUES (1, 1, 1, '2026-10-19 12:15:00', 12);   -- accepté
-- Réponse : WITH CHECK OPTION interdit d'insérer (ou modifier) par la vue une ligne qui ne
-- respecterait pas son WHERE. Salle 2 -> la ligne ne serait pas visible dans la vue -> refus.
-- Salle 1 -> respecte id_salle = 1 -> acceptée. annulee prend sa valeur par défaut (FALSE).
DELETE FROM seance WHERE debut = '2026-10-19 12:15:00' AND id_salle = 1;

Question D
-- Réponse : v_planning n'est PAS modifiable : elle contient un GROUP BY et des agrégats
(COUNT),
-- plus des colonnes calculées (CONCAT, places_restantes). Une ligne de la vue ne correspond
pas à
-- une seule ligne d'une table, MySQL ne saurait pas quoi modifier.
-- Intérêts de v_adherent_public :
-- - Sécurité : l'écran d'affichage n'a accès qu'à cette vue, jamais à la table adherent
--   (email, téléphone, date de naissance restent inaccessibles).
-- - RGPD : minimisation des données, on n'expose que le strict nécessaire (prénom + initiale),
--   la personne n'est pas identifiable par un visiteur.

-- Bonus : utilisateur accueil qui ne lit que v_planning
CREATE USER 'accueil'@'localhost' IDENTIFIED BY 'Accueil2026!';
GRANT SELECT ON vitasport.v_planning TO 'accueil'@'localhost';
-- Connecté en tant qu'accueil :
SELECT * FROM vitasport.v_planning;   -- OK
SELECT * FROM vitasport.paiement;     -- ERREUR 1142 : SELECT command denied to user 'accueil'
```